Artificial Intelligence

AI-2002

Lab 04

Uninformed Search Algorithms

National University of Computer & Emerging Sciences – NUCES – Karachi

Course Code: AI-2002 | Artificial Intelligence Lab

1. Objective 3

2. Uninformed Search 3

3. Types of searching Algorithms 4

3.1 Breadth First Search 4

3.2 Depth First Search 10

3.3 Depth Limited Search 14

3.4 Iterative Deepening Depth First Search 16

3.5 Uniform Cost Search 18

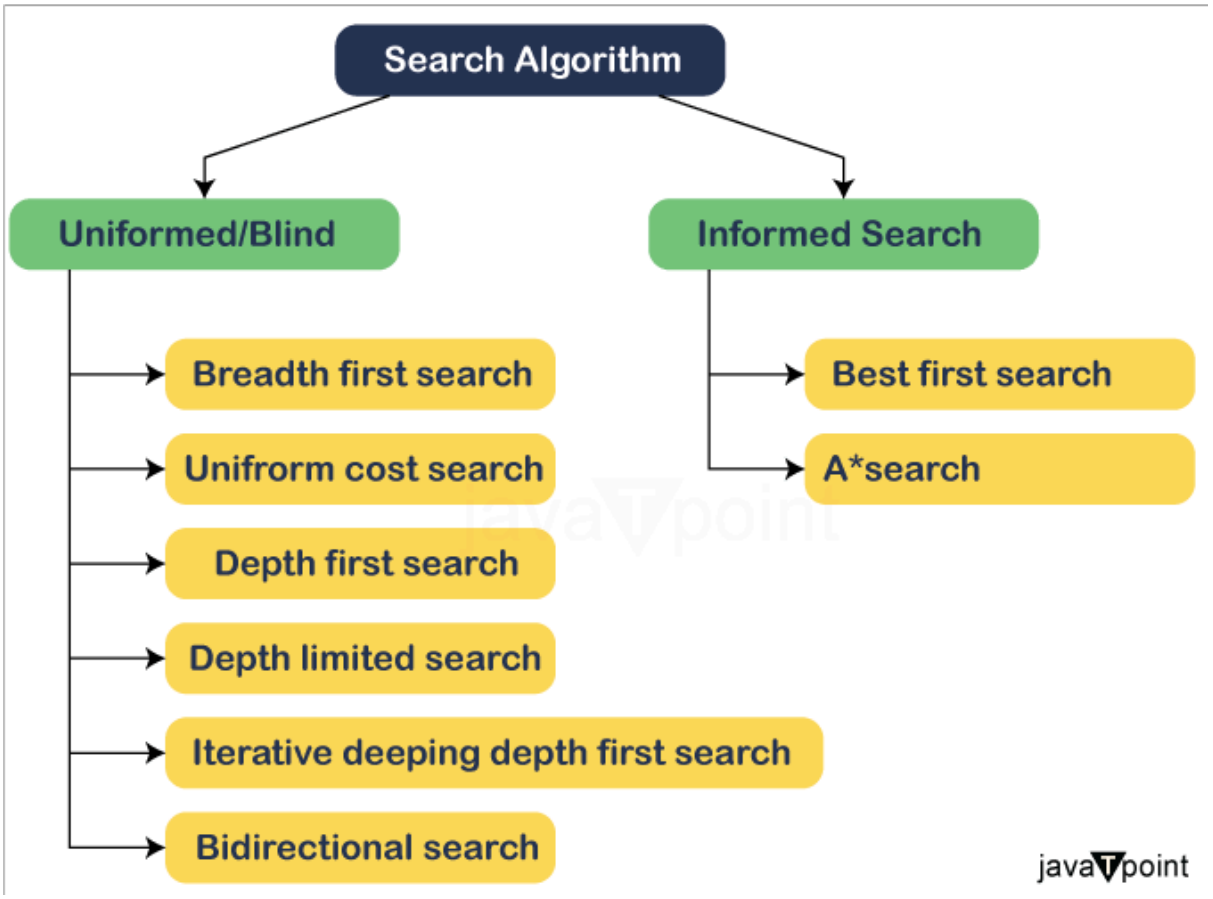
1. Objective

1. Introduction to Problem Solving by Searching
2. Implementing Uninformed/Blind Search Algorithms in Python

2. Uninformed Search

Uninformed search (also called blind search) is a type of search algorithm used in artificial intelligence and computer science to explore a problem space without any additional information about the problem other than its definition. These algorithms do not use any domain-specific knowledge or heuristics to guide the search. Instead, they systematically explore the search space until a solution is found.

3. Types of Searching Algorithms



3.1 Breadth first search

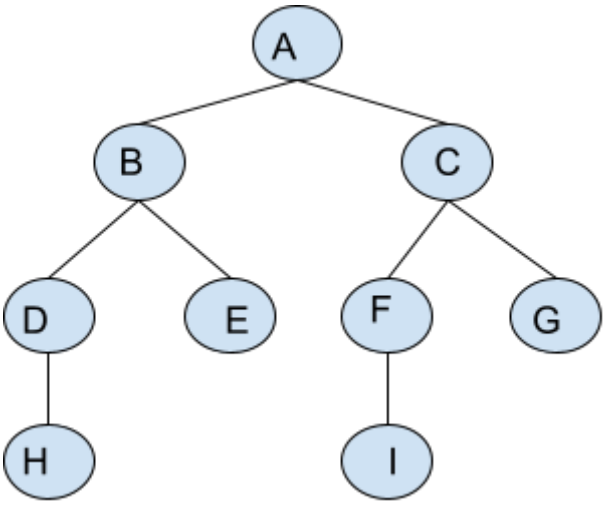
Example # 1

Step 1: Define the Environment (Tree)

This function implements the Breadth-First Search (BFS) algorithm to traverse a tree-like structure. It begins at the specified start node and explores the tree level by level, visiting all the nodes at the current depth before moving on to the next level. The search continues until the goal node is found or all possible nodes have been explored.

Tree Structure: The tree is represented as a dictionary, where each key is a parent node, and the value is a list of its children. For example:

```
# tree Representation
tree = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': ['H'],
    'E': [],
    'F': ['I'],
    'G': [],
    'H': [],
    'I': []
}
```



BFS Function: bfs(graph, start, goal)
The bfs function performs a breadth-first search on the given tree (graph) starting from the start node and searches for the goal node.

- **Parameters:**
  1. graph: The tree to be traversed.
  2. start: The node where the search begins.
  3. goal: The node that the algorithm is trying to find.
- **Process:**
  1. Initialize an empty list visited to track the nodes that have already been explored.
  2. Initialize a queue to manage the nodes to be explored.

3. The start node is added to both the visited list and the queue.
4. The search continues as long as there are nodes in the queue:
   - Dequeue a node from the front of the queue.
   - If the node matches the goal, the search stops and prints "Goal found!"
   - Otherwise, enqueue all unvisited neighbors of the current node.
5. If the goal node is found, the algorithm stops. If not, all reachable nodes are explored.

```
# BFS Function
def bfs(tree, start, goal):
    visited = []  # List for visited nodes
    queue = []    # Initialize a queue
    visited.append(start)
    queue.append(start)

    while queue:
        node = queue.pop(0)  # Dequeue
        print(node, end=" ")
        if node == goal:  # Stop if goal is found
            print("\nGoal found!")
            break
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)
```

Execution: To run the BFS search, define the start and goal nodes and then call the bfs function with the graph:

```
start_node = 'A'
goal_node = 'I'

# Run BFS
print("\nFollowing is the Breadth-First Search (BFS):")
bfs(tree, start_node, goal_node)
```

Working:

- The function starts at node 'A' and explores the tree in a breadth-first manner.
- Each node is visited in the order of their depth (level by level).
- Once the goal node ('I') is reached, the search stops and prints "Goal found!". If the goal is not found, it continues until all reachable nodes are explored.

We are now integrating the BFS code within the Agent and Environment terminology. In this setup, the agent is goal-oriented and uses the BFS algorithm to navigate through the environment, whether it's a tree, graph, or maze. The agent perceives its environment, determines if it has reached its goal, and if not, it uses the BFS strategy to explore the environment level by level, moving closer to the goal. The environment provides the necessary structure (tree, graph, maze) and supports the BFS search process, enabling the agent to find its way to the goal efficiently.

BFS Goal-Based Agent

In this section, we define a **Goal-Based Agent** that performs a BFS search to navigate through its environment. The agent formulates its goal and decides whether it has reached the goal based on the current percept. If the goal is not reached, the agent continues searching by invoking the BFS algorithm.

```
class GoalBasedAgent:
    def __init__(self, goal):
        self.goal = goal

    def formulate_goal(self, percept):
        if percept == self.goal:
            return "Goal reached"
        return "Searching"

    def bfs_search(self, graph, start, goal):
        visited = []  # List for visited nodes
        queue = []    # Initialize a queue

        visited.append(start)
        queue.append(start)

        while queue:
            node = queue.pop(0)  # Dequeue
            print(f"Visiting: {node}")
```

```
            if node == goal:  # Stop if goal is found
                return f"Goal {goal} found!"

            for neighbour in graph.get(node, []):
                if neighbour not in visited:
                    visited.append(neighbour)
                    queue.append(neighbour)

        return "Goal not found"

    def act(self, percept, graph):
        goal_status = self.formulate_goal(percept)
        if goal_status == "Goal reached":
            return f"Goal {self.goal} found!"
        else:
            return self.bfs_search(graph, percept, self.goal)
```

Environment Class

The **Environment class** represents the environment in which the agent operates. It can be a tree, graph, maze, or other structures. This class provides the bfs_search method that allows the agent to perform the breadth-first search to find its goal.

```
class Environment:
    def __init__(self, graph):
        #Initial graph/tree/maze
        self.graph = graph

    def get_percept(self, node):
        return node
```

Running the Agent

The run_agent function simulates the interaction between the agent and the environment. The agent starts at a given node, perceives its current state, and acts accordingly by performing the BFS search. The goal is to continue exploring until the goal is reached.

```
def run_agent(agent, environment, start_node):
    percept = environment.get_percept(start_node)
    action = agent.act(percept, environment.graph)  # Pass graph
to agent
    print(action)
```

In this example, the environment is a tree structure, and the agent is tasked with finding the node 'I' starting from 'A':

Execution: To run the BFS search, define the start and goal nodes and then call the bfs function with the tree/graph:

```
# Tree Representation
tree = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': ['H'],
    'E': [],
    'F': ['I'],
    'G': [],
    'H': [],
    'I': []
}

# Define Start and Goal Nodes
start_node = 'A'
goal_node = 'I'

# Create instances of agent and environment
agent = GoalBasedAgent(goal_node)
environment = Environment(tree)

# Run the agent
run_agent(agent, environment, start_node)
```

Example # 2 (Maze Grid Example)

The given Python code represents a maze as a graph and utilizes Breadth-First Search (BFS) to find a path from the start node (0,0) to the goal node (2,2). **The maze is a 2D grid where 1 represents an open path and 0 represents a blocked path.** The create_graph() function constructs a graph using an adjacency list, where each open cell is treated as a node, and edges connect nodes that have valid right or down movements. Once the graph is built, the bfs() function performs BFS traversal by exploring nodes layer by layer, ensuring the shortest path is found. A queue is used to process nodes in FIFO order, marking visited nodes to avoid repetition. The traversal starts from (0,0), visiting all reachable neighbors until the goal (2,2) is found, printing the traversal path along the way.

```
# Maze representation as a graph
maze = [
    [1, 1, 0],
    [1, 1, 0],
    [0, 1, 1]
]
# Directions for movement (right and down)
directions = [(0, 1), (1, 0)]  # (row, col)

# Convert maze to a graph (adjacency list representation)
def create_graph(maze):
    graph = {}
    rows = len(maze)
    cols = len(maze[0])
    for i in range(rows):
        for j in range(cols):
            if maze[i][j] == 1:  # If it's an open path
                neighbors = []
                for dx, dy in directions:
                    nx, ny = i + dx, j + dy
                    if 0 <= nx < rows and 0 <= ny < cols
and maze[nx][ny] == 1:
                        neighbors.append((nx, ny))
```

```
                graph[(i, j)] = neighbors
    return graph

# BFS Function using queue
def bfs(graph, start, goal):
    visited = []  # List for visited nodes
    queue = []    # Initialize queue

    visited.append(start)
    queue.append(start)

    while queue:
        node = queue.pop(0)  # FIFO: Dequeue from front
        print(node, end=" ")

        if node == goal:  # Stop if goal is found
            print("\nGoal found!")
            break

        for neighbour in graph[node]:  # Visit neighbors
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# Create graph from maze
graph = create_graph(maze)
# Define Start and Goal nodes
start_node = (0, 0)  # Starting point (0,0)
goal_node = (2, 2)   # Goal point (2,2)

# Run BFS
print("\nFollowing is the Breadth-First Search (BFS):")
bfs(graph, start_node, goal_node)
```

3.2 Depth first search

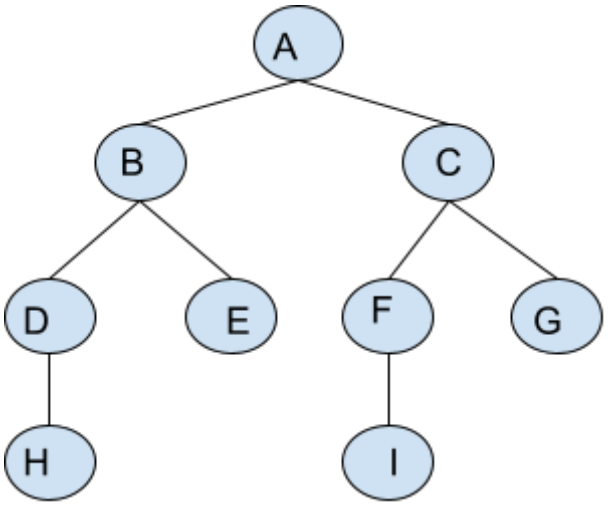
Depth-First Search (DFS) Algorithm

Step 1: Define the Environment (Tree)

This function implements the **Depth-First Search (DFS)** algorithm to traverse a tree-like structure. It begins at the specified start node and explores as deep as possible along each branch before backtracking. The search continues until the goal node is found or all possible nodes have been explored

Tree Structure: The tree is represented as a dictionary, where each key is a parent node, and the value is a list of its children. For example:.

```
# tree Representation
tree = {
'A': ['B', 'C'],
'B': ['D', 'E'],
'C': ['F', 'G'],
'D': ['H'],
'E': [],
'F': ['I'],
'G': [],
'H': [],
'I': []}
```



DFS Function:

The function dfs(graph, start, goal) takes in a graph (in this case, the tree), a start node, and a goal node, and performs a depth-first search to find the goal. The function utilizes a stack and a visited list to track the nodes to explore.

1. **Parameter:**
   The algorithm begins at the start node and initializes two data structures:
   - Visited list: To keep track of the nodes that have already been visited and prevent revisiting them.
   - Stack: A stack is used to store nodes that need to be explored. Since DFS is a Last In, First Out (LIFO) traversal, nodes are explored in depth-first order.

2. **Process:**
    - The algorithm first pushes the start node onto the stack and marks it as visited.
    - It then enters a loop where it pops nodes from the stack and explores them one by one:
        - If the popped node is the goal node, the search stops.
        - If the popped node has unvisited neighbors, they are pushed onto the stack for further exploration.
3. **Backtracking:**
    - If a node has no unvisited neighbors, the algorithm backtracks by popping the next node from the stack and continues the search from there.
    - This backtracking ensures that the algorithm explores deeper branches of the tree first before moving on to other branches.
4. **Termination:**
    - The algorithm terminates when the goal node is found, or if all possible nodes have been visited without finding the goal.

```
# DFS Function
def dfs(graph, start, goal):
    visited = []  # List for visited nodes
    stack = []    # Initialize stack
    visited.append(start)
    stack.append(start)

    while stack:
        node = stack.pop()  # LIFO: Pop from top
        print(node, end=" ")

        if node == goal:  # Stop if goal is found
            print("\nGoal found!")
            break

        for neighbour in reversed(graph[node]):  # Reverse to
maintain correct order
            if neighbour not in visited:
                visited.append(neighbour)
                stack.append(neighbour)
```

Execution: To run the DFS search, define the start and goal nodes and then call the dfs function with the graph:

```
# Define Start and Goal Nodes
start_node = 'A'
goal_node = 'I'

# Run DFS
print("\nFollowing is the Depth-First Search (DFS):")
dfs(graph, start_node, goal_node)
```

DFS Goal-Based Agent

In this example, we demonstrate how a **Goal-Based Agent** can operate within an **Environment** to search for a goal node using the **Depth-First Search (DFS)** algorithm

```
class GoalBasedAgent:
    def __init__(self, goal):
        self.goal = goal

    def formulate_goal(self, percept):
        if percept == self.goal:
            return "Goal reached"
        return "Searching"

    def dfs_search(self, graph, start, goal):
        visited = []  # List for visited nodes
        stack = []    # Initialize stack

        visited.append(start)
        stack.append(start)

        while stack:
            node = stack.pop()  # LIFO: Pop from top
            print(f"Visiting: {node}")

            if node == goal:  # Stop if goal is found
                return f"Goal {goal} found!"

            for neighbour in reversed(graph.get(node, [])):
                if neighbour not in visited:
```

```
                    visited.append(neighbour)
                    stack.append(neighbour)

        return "Goal not found"

    def act(self, percept, graph):
        goal_status = self.formulate_goal(percept)
        if goal_status == "Goal reached":
            return f"Goal {self.goal} found!"
        else:
            return self.dfs_search(graph, percept, self.goal)
```

Environment Class

The **Environment class** represents the environment in which the agent operates. It can be a tree, graph, maze, or other structures. This class provides the dfs_search method that allows the agent to perform the depth-first search to find its goal.

```
class Environment:
    def __init__(self, graph):
        self.graph = graph

    def get_percept(self, node):
        return node
```

Running the Agent

The run_agent function simulates the interaction between the agent and the environment. The agent starts at a given node, perceives its current state, and acts accordingly by performing the DFS search. The goal is to continue exploring until the goal is reached.

```
def run_agent(agent, environment, start_node):
    percept = environment.get_percept(start_node)
    action = agent.act(percept, environment.graph)  # Pass graph to
agent
    print(action)
```

Execution: To run the DFS search, define the start and goal nodes and then call the bfs function with the tree/graph:

```
# Tree Representation
tree = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': ['H'],
    'E': [],
    'F': ['I'],
    'G': [],
    'H': [],
    'I': []
}

# Define Start and Goal Nodes
start_node = 'A'
goal_node = 'I'

# Create instances of agent and environment
agent = GoalBasedAgent(goal_node)
environment = Environment(tree)

# Run the agent
run_agent(agent, environment, start_node)
```

3.3 Depth Limited search

Depth-Limited Search (DLS) is a variant of Depth-First Search (DFS) that restricts the search to a specified depth limit. DLS explores the nodes depth-first, but it will stop further exploration once the depth limit is reached. If the goal node is found within the depth limit, the search stops; otherwise, the algorithm backtracks and explores other possible paths within the limit.

This technique is particularly useful when the search space is large, and we want to avoid exploring unnecessary deep branches of the graph.

Define the Environment (Graph)

The environment is represented as an unweighted **graph**, where each key is a node, and its associated value is a list of neighboring nodes. The graph is structured as follows:

```
# Graph without weights for DLS
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': ['H'],
    'E': [],
    'F': ['I'],
    'G': [],
    'H': [],
    'I': []
}
```

DLS Function

The **Depth-Limited Search (DLS)** function performs a depth-first search with a maximum depth limit. This means that the algorithm will explore the graph, but it will not go deeper than the specified depth_limit.

Parameters:

1. **graph**: The graph structure to be searched.
2. **start**: The node where the search begins.
3. **goal**: The node the algorithm is trying to find.
4. **depth_limit**: The maximum depth the search can explore.

Process:

1. **Initialize a visited list** to track the nodes that have already been explored.
2. **Define a recursive dfs function** that takes a node and the current depth as arguments.
3. If the current depth exceeds the depth limit, return None (stop searching further).
4. **If the node matches the goal**, print the path to the goal.
5. **Explore each unvisited neighbor** of the current node by recursively calling dfs for each neighbor.

6. If no path to the goal is found within the limit, backtrack and explore other nodes.

```
# DLS Function
def dls(graph, start, goal, depth_limit):
    visited = []
    def dfs(node, depth):
        if depth > depth_limit:
            return None  # Limit reached
        visited.append(node)
        if node == goal:
            print(f"Goal found with DLS. Path: {visited}")
            # return visited
        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                path = dfs(neighbor, depth + 1)
                if path:
                    return path
        visited.pop()  # Backtrack if goal not found
        return "Goal not found in this depth"

    return dfs(start, 0)
```

Execution

To run the **DLS** search, we need to define the **start** node, **goal** node, and **depth_limit**. Then, we can call the dls function with these parameters.

```
# Run DLS with depth limit 3
dls(graph, 'A', 'I', 3)
```

Working of the DLS Algorithm

1. **Starting Point**: The algorithm starts at the start_node (A) and explores the graph depth-first.
2. **Depth Limitation**: The search is restricted to a maximum depth of depth_limit. If the depth exceeds this limit, the search will not continue.
3. **Goal Check**: If the goal_node (I) is found within the depth limit, the search stops, and the path to the goal is printed.

4. **Exploration Continuation**: If the goal is not found, the algorithm will backtrack and continue exploring other paths until the depth limit is reached.

3.4 Iterative Deepening Search

Iterative Deepening Search (IDS) is a search strategy that combines the benefits of **Depth-First Search (DFS)** and **Breadth-First Search (BFS)**. It performs repeated **Depth-Limited Searches (DLS)**, increasing the depth limit gradually until the goal is found or the maximum depth is reached.

How It Works:

1. **Start with a depth limit of 0.**
2. **Perform a Depth-Limited Search (DLS)** up to that depth.
3. **If the goal is found, return the result.** Otherwise, increase the depth and repeat.
4. **Continue until the goal is found** or the maximum depth is reached

```
tree = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': ['H'],
    'E': [],
    'F': ['I'],
    'G': [],
    'H': [],
    'I': []
}

def dls(node, goal, depth, path):
    if depth == 0:
        return False
    if node == goal:
        path.append(node)
        return True
    if node not in tree:
        return False
```

```
    for child in tree[node]:
        if dls(child, goal, depth - 1, path):
            path.append(node)  # Store nodes while
backtracking
            return True
    return False

def iterative_deepening(start, goal, max_depth):
    for depth in range(max_depth + 1):
        print(f"Depth: {depth}")
        path = []
        if dls(start, goal, depth, path):
            print("\nPath to goal:", " →
".join(reversed(path)))  # Print path correctly
            return
    print("Goal not found within depth limit.")

# Test Iterative Deepening
start_node = 'A'
goal_node = 'I'
max_search_depth = 5
iterative_deepening(start_node, goal_node, max_search_depth)
```

3.5 Uniform Cost Search

Uniform Cost Search (UCS) is a search algorithm that finds the least-cost path from the start node to the goal node in a weighted graph. Unlike Breadth-First Search (BFS), which explores all nodes at a given depth level before moving to the next level, UCS takes into account the cost of traversing edges. It always expands the node with the least cumulative cost, ensuring that the path found is the least expensive one.

In this example, we will implement UCS to find the least-cost path from the start node to the goal node in a weighted graph.

Define the Environment (Graph with Edge Costs)

We represent the graph with weighted edges, where each edge has a specific cost. Each node points to a dictionary of its neighbors and the corresponding costs.

```
# Graph with different edge costs for UCS
graph = {
    'A': {'B': 2, 'C': 1},
    'B': {'D': 4, 'E': 3},
    'C': {'F': 1, 'G': 5},
    'D': {'H': 2},
    'E': {},
    'F': {'I': 6},
    'G': {},
    'H': {},
    'I': {}
}
```

Step 2: **UCS Function Implementation**

The Uniform Cost Search (UCS) algorithm works by maintaining a frontier (a priority queue) that stores nodes along with their accumulated costs. The search expands the node with the lowest cost, ensuring an optimal path is found.

Parameters:

graph: The weighted graph to be traversed.

start: The node where the search begins.

goal: The node that the algorithm is trying to find.

Process:

Initialize the frontier with the start node and cost 0.

Initialize a visited set to track nodes that have already been explored.

Track the cost to reach each node using a dictionary (cost_so_far).

Maintain a path reconstruction dictionary (came_from) to reconstruct the path once the goal is reached.

The algorithm will:

- Sort the frontier by the accumulated cost to ensure the lowest cost node is expanded first.
- Expand the node, visiting its neighbors and updating the cost to reach each neighbor.
- If a neighbor offers a cheaper path, it is added to the frontier.

The algorithm continues until the goal is found or the frontier is empty.

```
# UCS Function with Frontier and Visited
def ucs(graph, start, goal):
    # Initialize the frontier with the start node and cost 0
    frontier = [(start, 0)]  # (node, cost)
    visited = set()  # Set to keep track of visited nodes
    cost_so_far = {start: 0}  # Cost to reach each node
    came_from = {start: None}  # Path reconstruction

    while frontier:
        # Sort frontier by cost, simulate priority queue
        frontier.sort(key=lambda x: x[1])
        # Pop the node with the lowest cost
        current_node, current_cost = frontier.pop(0)

        # If we've already visited this node, skip it
        if current_node in visited:
            continue
        # Mark the current node as visited
        visited.add(current_node)
        # If we reach the goal, reconstruct the path and
return
        if current_node == goal:
            path = []
            while current_node is not None:
                path.append(current_node)
                current_node = came_from[current_node]
            path.reverse()
            print(f"Goal found with UCS. Path: {path}, Total
Cost: {current_cost}")
            return

        # Explore neighbors
        for neighbor, cost in graph[current_node].items():
            new_cost = current_cost + cost
            if neighbor not in cost_so_far or new_cost <
cost_so_far[neighbor]:
```

```
                cost_so_far[neighbor] = new_cost
                came_from[neighbor] = current_node
                frontier.append((neighbor, new_cost))  # Add
to frontier

    print("Goal not found")
```

Execution

To run the **UCS** search, we need to define the **start** node, **goal** node. Then, we can call the UCS function with these parameters.

```
# Run UCS with updated costs, using frontier and visited
ucs(graph, 'A', 'I')
```

Lab tasks

1. A university campus is represented as a graph where each node is a building and edges represent walking paths connecting them directly. A Campus Navigation Agent needs to help a new student find the shortest number of steps (in terms of number of paths taken, not distance) from the Main Gate to the Library. Represent the campus as a dictionary-based graph with at least 9 buildings, and implement a Goal-Based Agent that uses BFS to explore the campus level by level starting from the Main Gate until it reaches the Library. The agent should print each building as it is visited, and once the Library is found, it should print the confirmation message along with the order in which buildings were explored.

2. A file system on a computer is structured as a tree, where each folder can contain multiple subfolders, and one specific subfolder deep within the structure contains a corrupted file that needs to be located. Represent the folder structure as a dictionary-based tree with at least 9 folders. Implement a Goal-Based Agent that uses DFS to explore the folder structure as deep as possible along each branch before backtracking, searching for the folder containing the corrupted file. The agent should print each folder as it is visited and confirm once the target folder is found, or report that no such folder exists if the search space is exhausted.

3. A social media platform wants to explore a user's network of connections to find a specific person, but only within a limited number of connection "hops" (e.g., friends of friends up to 3 levels deep) to avoid an overwhelming search. Represent the connection network as a dictionary-based graph with at least 9 users. Implement a Depth-Limited Search function that takes the graph, a starting user, a target user, and a depth limit as parameters, and explores the network depth-first while never exceeding the specified hop limit. If the target user is found within the limit, print the path taken to reach them; if not, print a message indicating the person could not be found within the given number of hops.